

ConverterLib Unit Testing using NUnit and Moq

Introduction

This experiment explains how to write automated unit test cases for the ConverterLib project using NUnit and the Moq framework. The USDToEuro() method depends on an external exchange rate service. Instead of using the real service, a mock object is created using Moq to isolate the business logic during testing.

Objectives

- Learn automated unit testing using NUnit.
- Understand how the Moq framework isolates external dependencies.
- Learn how to create testable code using mock objects.
- Verify the USDToEuro() method using Assert.That().

Implementation

Step 1: Create a Class Library project named ConverterLib.Tests in the same solution.

Step 2: Rename the default class to ConverterTests.cs.

Step 3: Add the reference of ConverterLib to the test project.

Step 4: Install NUnit, NUnit3TestAdapter, and Moq using NuGet Package Manager.

Step 5: Create a mock object for the IDollarToEuroExchangeRateFeed interface using the Moq framework.

Step 6: Configure the mock object to return a predefined exchange rate instead of calling the real service.

Step 7: Pass the mock object to the Converter class using dependency injection.

Step 8: Write unit test methods to verify the USDToEuro() method with different input values.

Step 9: Use Assert.That() to compare the expected and actual conversion results.

Step 10: Build the solution and run all unit tests.

Step 11: Modify the Converter class intentionally to break one test case and rerun the tests to observe the failed result.

Execution

Paste the screenshot of the successful NUnit Test Explorer or terminal output here.

```
1 using NUnit.Framework;
2 using ConverterLib;
3 using Moq;
4
5 namespace ConverterLib.Tests
6 {
7     [TestFixture]
8     public class ConverterTests
9     {
10         private Mock<IDollarToEuroExchangeRateFeed> mockFeed;
11         private Converter converter;
12
13         [Setup]
14         public void Setup()
15         {
16             // ...
17         }
18     }
19 }
```

```
PS C:\Users\VASUS\Downloads\ConverterLib.Tests> dotnet add ConverterLib.Tests reference ConverterLib\ConverterLib.csproj
>> dotnet add ConverterLib.Tests package NUnit
>> dotnet add ConverterLib.Tests package NUnit3TestAdapter
>> dotnet add ConverterLib.Tests package Microsoft.NET.Test.Sdk
>> dotnet add ConverterLib.Tests package Moq
Info : CACAE https://api.nuget.org/v3-vulnerabilities/2026.07.16.05.27/2026.07.17.11.27.26/vulnerability.update.json
Info : Package "Moq" is compatible with all the specified frameworks in project 'C:\Users\VASUS\Downloads\ConverterLib.Tests\ConverterLib.Tests\ConverterLib.Tests.csproj'.
Info : PackageReference for package 'Moq' version '4.20.72' added to file 'C:\Users\VASUS\Downloads\ConverterLib.Tests\ConverterLib.Tests\ConverterLib.Tests.csproj'.
Info : Writing assets file to disk. Path: C:\Users\VASUS\Downloads\ConverterLib.Tests\ConverterLib.Tests\obj\project.assets.json
log : Restored C:\Users\VASUS\Downloads\ConverterLib.Tests\ConverterLib.Tests\ConverterLib.Tests.csproj (in 242 ms).
PS C:\Users\VASUS\Downloads\ConverterLib.Tests> dotnet test
Restore complete (0.7s)
ConverterLib.Tests net8.0 succeeded with 1 warning(s) (0.9s) - bin\Debug\net8.0\ConverterLib.Tests.dll
C:\Program Files\dotnet\sdks\10.0.301\Microsoft.Common.CurrentVersion.targets(2205,5): warning MSB90008: The referenced project ..\ConverterLib\ConverterLib.csproj does not exist.
NUnit Adapter 4.5.0.0: Test execution started
Running all tests in C:\Users\VASUS\Downloads\ConverterLib.Tests\bin\Debug\net8.0\ConverterLib.Tests.dll
NUnit3TestExecutor discovered 4 of 4 NUnit test cases using Current Discovery mode, Non-Explicit run
NUnit Adapter 4.5.0.0: Test execution complete
ConverterLib.Tests test net8.0 succeeded (1.5s)
Test summary: total: 4, failed: 0, succeeded: 4, skipped: 0, duration: 1.5s
Build succeeded with 1 warning(s) in 3.9s
PS C:\Users\VASUS\Downloads\ConverterLib.Tests>
```

Expected Result

The USDToEuro() method should return the expected Euro value based on the mocked exchange rate. All unit test cases should pass when the implementation is correct. After modifying the source code, one or more test cases should fail, confirming that the tests are working correctly.

Conclusion

This experiment helped in understanding how Moq can be used to replace external services during unit testing. Using mock objects improved test reliability, reduced dependency on external systems, and made the Converter class easier to test.